

Exercises in Numerical Modeling of Glaciers

The [notes](#) and the [slides](#) are background. Abbreviations: SKE = surface kinematical equation, SIA = shallow ice approximation, CFL = Courant-Friedrichs-Lewy.

PAPER EXERCISES

1. Sketch a smooth ice surface $z = s(x)$ in the planar case. Convince yourself that the vector $\mathbf{n}_s = (-\frac{\partial s}{\partial x}, 1)$ is normal to the surface and upward. What is the equivalent formula when $z = s(x, y)$, i.e. in 3D reality?
2. Assume $a = 0$ and $w = 0$, so that the SKE is simply $\frac{\partial s}{\partial t} + u \frac{\partial s}{\partial x} = 0$. Assume that u is not zero. For grid spacing $\Delta t > 0, \Delta x > 0$, rewrite the forward-time centered-space scheme from the slides as $s_j^{\text{new}} = c_- s_{j-1} + c_0 s_j + c_+ s_{j+1}$. Show that the coefficients c_-, c_0, c_+ sum to one, but that at least one coefficient is negative. Is $c_- A + c_0 B + c_+ C$ an average of A, B, C ? Argue not; this “average” can be less than any of A, B, C .
3. Continuing the assumptions in 2, write down the upwind scheme, from the slides, for both the $u_j \geq 0$ and $u_j \leq 0$ cases. Show that if the CFL condition $|u_j| \frac{\Delta t}{\Delta x} \leq 1$ applies then the new surface value is a true average of the old values. For instance, when $u_j \geq 0$ show that $s_j^{\text{new}} = c_- s_{j-1} + c_0 s_j$ where c_-, c_0 are nonnegative and sum to one. What will happen if Δt is too large, that is, if the CFL condition is violated?
4. Practice using the Leibniz rule for differentiating integrals, namely this rule

$$\frac{d}{dx} \left(\int_{g(x)}^{f(x)} k(x, y) dy \right) = f'(x) k(x, f(x)) - g'(x) k(x, g(x)) + \int_{g(x)}^{f(x)} \frac{\partial k}{\partial x}(x, y) dy,$$

by finding $\frac{\partial}{\partial x} \left(\int_{b(t,x)}^{s(t,x)} u(t, x, z) dz \right)$. This will be used next.

5. In the planar case, derive the mass continuity equation ($\frac{\partial H}{\partial t} + \frac{\partial}{\partial x}(\bar{u}H) = a$) from the SKE ($\frac{\partial s}{\partial t} + u \frac{\partial s}{\partial x} = a + w$), using the incompressibility of ice ($\frac{\partial u}{\partial x} + \frac{\partial w}{\partial z} = 0$). Here $\bar{u}(t, x) = \frac{1}{H(t,x)} \int_{b(t,x)}^{s(t,x)} u(t, x, z) dz$ is the vertically-averaged ice velocity; note $\bar{u}H = \int_b^s u dz$ is called the *ice flux*. Start the derivation by computing $\frac{\partial}{\partial x}(\bar{u}H)$. Assume the bed is not moving ($\frac{\partial b}{\partial t} = 0$) and the ice is not sliding ($u(t, x, b(x)) = 0$) or penetrating ($w(t, x, b(x)) = 0$).
6. Write out the details of the slab-on-a-slope calculation from the slides. Thereby derive the ($n = 3$) velocity formula $u(z) = u_0 + \frac{1}{2} A (\rho g \sin \alpha)^3 (H^4 - (H - z)^4)$. Now add in x -dependence, to see the velocity formula in the SIA model.
7. (Do with a friend.) Sketch a hypothetical planar glacier shape, with smooth surface and bed. Sketch what you think the non-sliding SIA formulas will generate for the surface values of the horizontal (u) and vertical (w) velocity components. Can you tell how the non-sliding Stokes model would change your pictures?
8. (Do with a friend.) For the ice thickness $H = s - b$, what inequality is equivalent to “ $s \geq b$ ”? Which other geophysical fluid layer problems, in cryosphere contexts and outside, have the same inequalities? What (physically) determines the shapes of these fluid layers, and their margins and termini? (Remember surface mass processes.)

COMPUTER EXERCISES

These exercises are based on Python codes in the `py/` directory of `github.com/bueler/mccarthy`; see the `README.md`. When you modify a code, it is best to copy to a new source file, and modify that. Keep the original code in pristine condition to check how things are supposed to work.

9. Modify `surface.py` to systematically test its (conditional) stability. Specifically, vary the grid spacings Δt and Δx , but also vary the velocity formula. Confirm that the CFL condition $|u_j| \frac{\Delta t}{\Delta x} \leq 1$ predicts which cases are stable and unstable.
10. Modify `surface.py` to use the forward-time centered-space scheme, and confirm that you get instabilities for essentially any $\Delta t, \Delta x$. (Then revert to upwind.)
11. Modify `surface.py` to show that the scheme can generate surfaces which fail $s \geq b$, i.e. which violate *admissibility*. (Increase the ablation rate to cause this.) Then modify the scheme so that it maintains admissibility via *truncation*. That is, compute a preliminary surface elevation $\tilde{s}_j^{\text{new}}$ and then apply $s_j^{\text{new}} = \max\{\tilde{s}_j^{\text{new}}, b_j\}$.
12. Modify `surface.py` to solve a case where the horizontal velocity depends on time and space: $u = u(t, x)$. Then modify the time-stepping scheme so that the time step is determined via the CFL condition. That is, at each time step, find the largest horizontal speed and adjust Δt for the next step accordingly.

Comment. Any serious numerical model for glacier geometry should at least have the mechanisms in 11 and 12. The surface mass balance could then safely come from any energy balance sub-model, and the surface velocities from any stress balance sub-model. (See 16 below.) However, numerical stability will not be *guaranteed* by the CFL above, *because ice flows downhill*. The precise stability condition is mostly an open question.

13. Modify `surface.py` so that the data a, u, w can be read from a file. Check that it runs as before by loading the same values as in the unmodified code.
14. Modify the ice geometry in `shallow.py` to a case where the glacier terminates. By trying different fractional power shapes for the terminus (margin), show that in some cases the horizontal SIA surface velocity goes to zero as one approaches the terminus, and in some cases not. (Look up “Vialov glacier profile” for a bad case.) Explain what is going on in terms of the powers which appear in the SIA formulas for surface velocity.
15. Modify `dome.py` to work with a non-flat bed, and choose a test case for plotting. Noting that the Halfar exact solution requires a flat bed, how would you verify that your modifications are correct?
16. In a suitable textbook, read about *implicit time-stepping*, and perhaps also about the *method of lines*. Modify `dome.py` to use an implicit scheme. Do you see benefits? What should replace truncation—see 11, or the corresponding code in `dome.py`—in such an implicit scheme?